

Titre professionnel RNCP 37827

Développeur en intelligence artificielle

Niveau 6, équivalent à Bac +3/4



livrable E5

Surveillance et résolution d'incidents

Déboguer une application de classification d'images satellite

Passage par la voie de la formation — parcours de 19 mois achevé en mars 2026

Alexandre Laugier

mars 2026

Vendredi 20 mars 2026

Table des matières

1	Contexte et description de l'application	3
1.1	Objectif	3
1.2	Stack technique	3
2	Identification et correction des bugs [C21]	3
2.1	Bug 1 — Redimensionnement absent	3
2.2	Bug 2 — Double normalisation	4
2.3	Flux de traitement corrigé	5
3	Validation par tests automatisés et pipeline CI/CD [C21]	5
3.1	Stratégie de test	5
3.2	Résultats	5
3.3	Pipeline CI/CD	5
4	Observabilité : journalisation, alerting et monitoring [C20]	6
4.1	Métriques surveillées et seuils d'alerte	6
4.2	Journalisation	6
4.3	Alerting	7
4.4	Monitoring	7
5	Feedback loop, déploiement et perspectives [C20, C21]	7
5.1	Versionnement et traçabilité des correctifs (C21)	7
5.2	Collecte des feedbacks	7
5.3	Déploiement sur Render	7
5.4	Stratégie de ré-entraînement sans fuite de données	8
	Références	9
	Glossary	10
	Annexes	11

Compétences RNCP 37827 couvertes par ce livrable

Code	Compétence	Section(s) concernée(s)
C20	Surveiller une application IA : monitoring, journalisation, feedback loop, détection automatique d'incidents	§ 4 — Observabilité
C21	Résoudre les incidents techniques : diagnostic, correction, documentation, versionnement Git	§ 2 — Bugs & correctifs, § 3 — Tests & CI/CD

1 Contexte et description de l'application

1.1 Objectif

L'application est une interface web de **classification d'images satellite** développée avec **Flask**. Elle permet à un utilisateur de soumettre une image et d'obtenir une prédiction parmi quatre classes : `desert`, `forest`, `meadow` et `mountain`. L'inférence repose sur un modèle **CNN** pré-entraîné au format **Keras**. Un mécanisme de **Feedback loop** utilisateur alimente une base **MongoDB Atlas** pour permettre un futur ré-entraînement du modèle.

1.2 Stack technique

La stack complète est détaillée en Annexe F. En résumé : backend Flask 3.1.2, inférence Keras 3.11.3 + PyTorch CPU, persistance MongoDB Atlas, monitoring Flask-MonitoringDashboard / **SQLite**, 15 tests `unittest`, déploiement **Gunicorn** + **Render** (PaaS).

Accessibilité documentaire. Ce document respecte les recommandations d'accessibilité numérique de l'association Valentin Haüy (AVH) et d'Atalan – AcceDe : structure hiérarchisée par titres numérotés, légendes systématiques sous les figures et tableaux, symboles non-couleur dans les tableaux de résultats, police sans empattement (Helvetica 12 pt), contraste texte/fond conforme, sommaire cliquable et navigation par sections.

2 Identification et correction des bugs [C21]

Synthèse des incidents — compétence C21			
Bug	Symptôme	Type	Impact
#1	<code>ValueError</code> : forme image incompatible	Bloquant	Application inutilisable ; erreur visible en logs
#2	Prédictions identiques pour toutes images	Silencieux	Modèle dégradé ; aucune exception levée

Deux bugs indépendants ont été identifiés dans la fonction `preprocess_from_pil()` de `app/app.py`. Le premier provoque une erreur bloquante visible dans les logs ; le second est silencieux mais tout aussi critique : il dégrade les prédictions sans lever d'exception.

2.1 Bug 1 — Redimensionnement absent

Symptôme

Au lancement d'une prédiction sur une image uploadée, le serveur lève l'exception suivante :

```
ValueError: Input 0 of layer "functional_1" is incompatible with the layer:
expected shape=(None, 224, 224, 3), found shape=(1, 600, 600, 3)
```

Le modèle Keras attend un tenseur d'entrée de forme (None, 224, 224, 3), mais reçoit un tenseur (1, 600, 600, 3) correspondant à l'image originale non redimensionnée.

Cause racine

La fonction `preprocess_from_pil()` convertit l'image en RGB et la normalise, mais **omet l'étape de redimensionnement**. L'image est donc transmise au modèle à sa taille d'origine. Le code fautif est reproduit en Annexe A.

Correctif appliqué

La correction s'effectue en deux étapes :

1. **Extraction dynamique des dimensions** attendues par le modèle lors du chargement, pour éviter toute valeur codée en dur :

```
model = keras.saving.load_model(MODEL_PATH, compile=False)
MODEL_HEIGHT, MODEL_WIDTH = input_shape[1], input_shape[2]
```

2. **Ajout du redimensionnement** via `Image.Resampling.LANCZOS` [3] (filtre haute qualité, conforme aux recommandations Pillow ≥ 9.1).

2.2 Bug 2 — Double normalisation

Symptôme

Après correction du redimensionnement, les **prédictions sont identiques pour toutes les images** : le modèle retourne systématiquement la même classe quelle que soit l'entrée. Aucune exception n'est levée — le bug est entièrement **silencieux**.

Cause racine

L'inspection via `model.layers[2].get_config()` révèle une couche `Rescaling` intégrée (`scale=1/255`). La fonction `preprocess_from_pil()` appliquait en plus une division `/ 255.0`, soit une double normalisation : les pixels étaient divisés par $255^2 = 65\,025$, produisant des valeurs $\approx 10^{-5}$. Le CNN recevait un signal quasi nul, rendant toutes les activations uniformes et les prédictions dégénérées.

Correctif appliqué

Suppression du `/ 255.0` dans `preprocess_from_pil()` : le tenseur est désormais transmis dans `[0, 255]`, la couche `Rescaling` du modèle assurant seule la normalisation. Le tableau de synthèse avant/après est disponible en Annexe A.

2.3 Flux de traitement corrigé

Le schéma ci-dessous illustre le flux complet après application des deux correctifs : le chemin de prédiction (haut) traite l'image redimensionnée et non sur-normalisée, tandis que le flux de monitoring (bas, orange) collecte les métriques en parallèle sans impact sur les performances.

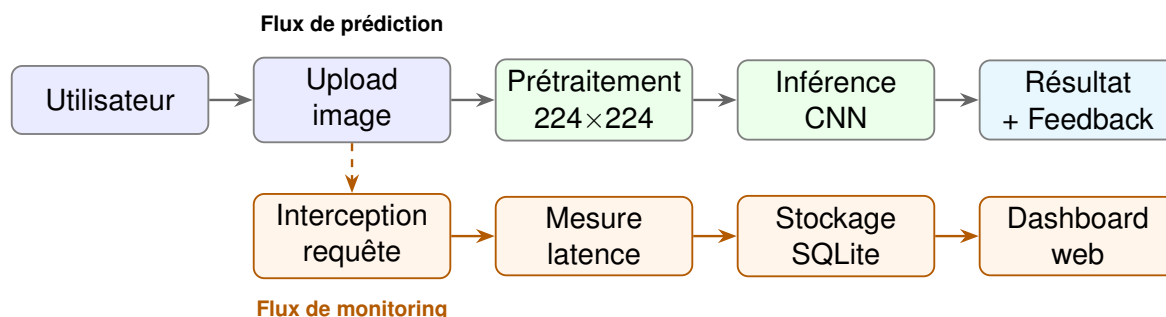


FIGURE 1 – Flux de traitement corrigé : prédiction (haut) et monitoring en parallèle (bas, orange pointillé).

3 Validation par tests automatisés et pipeline CI/CD [C21]

3.1 Stratégie de test

La suite de tests est organisée en trois classes unittest : `TestImageClassificationApp` couvre le flux de traitement des images (validation des extensions, prétraitement, encodage, routes Flask, feedback) ; `TestModelIntegration` vérifie la compatibilité entre le prétraitement et l'interface du modèle ; `TestPreprocessNormalization` documente le Bug 2 et valide son correctif. Les tests utilisent des **Mocks** pour simuler le modèle Keras sans dépendre du fichier `.keras`.

Le test critique (`test_preprocess_from_pil_shape_correction`) soumet une image synthétique 600×600 et vérifie que la sortie a bien la forme (1, 224, 224, 3) attendue par le modèle. Son code est reproduit en Annexe B.

3.2 Résultats

La suite CI passe **13/13 tests** en 18,4s sur Python 3.11 et 3.12 ; 2 tests d'intégration nécessitant le vrai modèle Keras sont sautés en CI et s'exécutent localement lors de la démonstration. Le tableau détaillé par catégorie est disponible en Annexe G.

3.3 Pipeline CI/CD

Les tests sont intégrés dans un pipeline GitHub Actions [4] déclenché à chaque push sur la branche `main`. Le workflow `.github/workflows/main.yml` exécute la suite en parallèle sur

Python 3.11 et 3.12, puis déclenche automatiquement le déploiement sur Render si tous les tests passent (`needs: test`).

Deux adaptations techniques ont été nécessaires pour le CI :

1. **Installation explicite de torch CPU** avant `requirements.txt`, car Keras tente d'importer torch dès son initialisation :

```
pip install torch --index-url https://download.pytorch.org/whl/cpu
```

2. **Correction d'une assertion de test** incohérente avec le correctif Bug 2 : la vérification `valeurs ∈ [0, 1]` a été remplacée par `valeurs ∈ [0, 255]`, conformément à la délégation de la normalisation à la couche Rescaling.

4 Observabilité : journalisation, alerting et monitoring [C20]

4.1 Métriques surveillées et seuils d'alerte

Le tableau suivant liste les métriques instrumentées, leurs seuils d'alerte et les outils associés, conformément aux exigences de la compétence C20.

Métrique	Seuil warning	Seuil critique	Outil
Latence /predict	> 500 ms	> 2 000 ms	Flask-MonitoringDashboard / SQLite
Taux d'erreur HTTP 5xx	> 1 %	> 5 %	logging.ERROR + SMTPHandler
Taux d'erreur HTTP 4xx	> 10 %	> 25 %	logging.WARNING
Disponibilité application	< 99 %	< 95 %	Render health-check + alerting SMTP
Rate limiting déclenché	> 5 req/s	> 10 req/s	Décorateur @rate_limit
Connexion MongoDB Atlas	Timeout	Échec persistant	logger.error() + SMTPHandler

TABLE 1 – Métriques surveillées, seuils d'alerte et outils associés (C20).

4.2 Journalisation

La journalisation est assurée par la bibliothèque standard `logging` [8], configurée avec deux *handlers* : un fichier `app.log` et la sortie console. Trois niveaux sont utilisés selon la gravité : `logger.info()` pour l'état des services (ex. connexion MongoDB établie), `logger.warning()` pour les situations dégradées sans arrêt du service (ex. variable d'environnement absente), et `logger.error()` pour les exceptions avec traceback (ex. erreur de connexion à Atlas).

4.3 Alerting

En production, un système d'alerte email est activé via `SMTPHandler` (bibliothèque `logging.handlers`). Tout log de niveau `ERROR` ou supérieur déclenche automatiquement l'envoi d'un email avec sujet « ERREUR CRITIQUE -- App Classification » vers l'adresse définie dans la variable d'environnement `ALERT_EMAIL_RECIPIENT`. Les credentials SMTP sont chargés depuis les variables d'environnement, jamais codés en dur.

4.4 Monitoring

`Flask-MonitoringDashboard` est monté sur le endpoint `/dashboard`. Il intercepte automatiquement chaque requête HTTP et enregistre dans une base SQLite locale : le temps de réponse par endpoint, le code HTTP retourné, l'utilisation horaire et la comparaison multi-version. Des scripts Python dédiés (`monitoring/`) permettent d'extraire ces métriques pour une analyse hors ligne (détection de régressions, pics de latence).

Un décorateur `@rate_limit` assure le **Rate limiting** de l'endpoint `/predict` (10 req/min). Il est activé **uniquement en production** via `FLASK_ENV=production`, afin d'éviter les délais artificiels en développement et de ne pas fausser les métriques du dashboard. Le code correspondant est disponible en Annexe H.

5 Feedback loop, déploiement et perspectives [C20, C21]

5.1 Versionnement et traçabilité des correctifs (C21)

Les deux correctifs sont versionnés et accessibles sur le dépôt GitHub du projet : <https://github.com/alaugier/flask-image-classifier-debug>

5.2 Collecte des feedbacks

Après chaque prédiction, l'utilisateur peut corriger le label via un appel AJAX (`POST /feedback`). Le backend valide les champs reçus et insère dans MongoDB Atlas un document structuré contenant l'image encodée en base64, le label prédit avec sa confiance, le label corrigé par l'utilisateur, un booléen `is_correct` et la version du modèle ayant effectué la prédiction.

5.3 Déploiement sur Render

L'application est déployée en continu sur Render (PaaS, plan gratuit) à l'adresse <https://flask-image-classifier-debug.onrender.com>. Le déploiement est déclenché **automatiquement** après validation des tests CI — aucune intervention manuelle n'est requise. La build installe `torch CPU` en amont de `requirements.txt` pour éviter le téléchargement de la version `CUDA` (≈2 Go). Les variables sensibles (`KERAS_BACKEND`, `MONGO_URI`, credentials SMTP) sont injectées via les variables d'environnement Render, jamais stockées dans le dépôt. Des captures d'écran du dashboard Render et des logs de déploiement sont disponibles en Annexes D et E.

5.4 Stratégie de ré-entraînement sans fuite de données

Un script `cron` quotidien déclenche un pipeline de ré-entraînement dès qu'un seuil de `feedbacks corrigés` est atteint (ex. 100 observations). Le jeu de test initial est **gelé** : seul l'ensemble d'entraînement est étendu avec les nouvelles données MongoDB, garantissant la comparabilité des métriques entre versions et éliminant tout risque de [Data leakage](#) — approche [MLOps](#) standard.

Références

- [1] Bogdan ANTONY et Patrick VISSER. *Flask Monitoring Dashboard – Automatically monitor your Flask web service*. Version 3.1.0. Consulté en mars 2026. 2024. URL : <https://flask-monitoringdashboard.readthedocs.io>.
- [2] François CHOLLET et al. *Keras – Deep Learning for humans*. Version 2.13.1. Consulté en mars 2026. 2024. URL : <https://keras.io>.
- [3] Alex CLARK et CONTRIBUTORS. *Pillow – Python Imaging Library (Fork)*. Version 10.x. Consulté en mars 2026. 2024. URL : <https://pillow.readthedocs.io>.
- [4] GITHUB, INC. *GitHub Actions — Automate your workflow*. Consulté le 17 mars 2026. 2024. URL : <https://docs.github.com/en/actions>.
- [5] GUNICORN CONTRIBUTORS. *Gunicorn – Python WSGI HTTP Server for UNIX*. Consulté en mars 2026. 2024. URL : <https://gunicorn.org>.
- [6] MONGODB, INC. *MongoDB Atlas Documentation*. Consulté en mars 2026. 2024. URL : <https://www.mongodb.com/docs/atlas/>.
- [7] PALLETS PROJECTS. *Flask Documentation – A lightweight WSGI web application framework*. Version 2.3.3. Consulté en mars 2026. 2024. URL : <https://flask.palletsprojects.com>.
- [8] PYTHON SOFTWARE FOUNDATION. *logging – Logging facility for Python*. Python 3.11. Consulté en mars 2026. 2024. URL : <https://docs.python.org/3/library/logging.html>.
- [9] PYTHON SOFTWARE FOUNDATION. *unittest – Unit testing framework*. Python 3.11. Consulté en mars 2026. 2024. URL : <https://docs.python.org/3/library/unittest.html>.

Glossaire

CNN

Convolutional Neural Network : réseau de neurones convolutif, architecture de référence pour la classification d'images.

Data leakage

Fuite de données : contamination du jeu de test par des données d'entraînement, biaisant les métriques d'évaluation du modèle.

Feedback loop

Boucle de rétroaction : mécanisme de collecte des corrections utilisateurs pour améliorer le modèle de façon itérative (approche MLOps).

Flask

Micro-framework Python pour le développement d'applications web et d'API REST (WSGI).

Gunicorn

Green Unicorn : serveur HTTP WSGI Python pour environnements de production Unix.

Keras

API de haut niveau pour la construction et l'entraînement de réseaux de neurones, s'appuyant sur TensorFlow ou PyTorch.

MLOps

Ensemble de pratiques (DevOps appliqué au ML) visant à industrialiser le cycle de vie des modèles d'intelligence artificielle.

Mock

Objet simulé utilisé en test unitaire pour remplacer une dépendance réelle (ex. le modèle Keras) sans la charger en mémoire.

MongoDB Atlas

Service cloud de base de données NoSQL orientée documents, utilisé ici pour la persistance des feedbacks utilisateurs.

PaaS

Platform as a Service : plateforme cloud gérant l'infrastructure, permettant de déployer une application sans administrer de serveur physique.

Rate limiting

Limitation du nombre de requêtes acceptées par unité de temps sur un endpoint, pour protéger contre les abus et les attaques par déni de service.

Render

Plateforme PaaS de déploiement continu hébergeant l'application Flask en production.

SQLite

Base de données relationnelle embarquée (fichier unique), utilisée par Flask-MonitoringDashboard pour stocker les métriques de performance.

Annexes

A — Fonction preprocess_from_pil() : avant et après correctifs

Version originale (deux bugs) :

```
def preprocess_from_pil(pil_img: Image.Image) -> np.ndarray:
    img = pil_img.convert("RGB")
    # BUG 1 : pas de resize -> ValueError shape incompatible
    # BUG 2 : normalisation / 255.0 alors que le modele contient
    #         deja une couche Rescaling(1/255) -> double normalisation
    img_array = np.asarray(img, dtype=np.float32) / 255.0
    img_array = np.expand_dims(img_array, axis=0)
    return img_array
```

Version finale corrigée (deux correctifs intégrés) :

```
# Chargement du modele et extraction dynamique des dimensions
model = keras.saving.load_model(MODEL_PATH, compile=False)
input_shape = model.input_shape          # ex. (None, 224, 224, 3)
MODEL_HEIGHT, MODEL_WIDTH = input_shape[1], input_shape[2]
# Note : model.layers[2] est une couche Rescaling(scale=1/255)
# => la normalisation est deja integree dans le modele

def preprocess_from_pil(pil_img: Image.Image) -> np.ndarray:
    """Prepare une image PIL pour l'inference Keras.

    Args:
        pil_img: Image PIL source, taille quelconque.

    Returns:
        np.ndarray de forme (1, MODEL_HEIGHT, MODEL_WIDTH, 3),
        dtype float32, valeurs dans [0, 255] -- la normalisation
        est deleguee a la couche Rescaling du modele.
    """
    # 1. Conversion RGB
    img = pil_img.convert("RGB")
    # 2. CORRECTIF 1 : redimensionnement aux dimensions du modele
    img = img.resize((MODEL_WIDTH, MODEL_HEIGHT),
                     Image.Resampling.LANCZOS)
    # 3. Conversion en array float32
    # CORRECTIF 2 : suppression du / 255.0
    # (la couche Rescaling du modele s'en charge)
    img_array = np.asarray(img, dtype=np.float32)
    # 4. Ajout de l'axe batch
    img_array = np.expand_dims(img_array, axis=0)
    return img_array    # shape : (1, 224, 224, 3), valeurs in [0, 255]
```

B — Tests unitaires : vérification du redimensionnement

```
1 class TestImageClassificationApp(unittest.TestCase):
2
3     def test_preprocess_from_pil_shape_correction(self):
4         """Test critique : le preprocessing doit redimensionner
5         toute image a la forme attendue par le modele."""
6         test_img = Image.new('RGB', (600, 600), color='blue')
7         processed = preprocess_from_pil(test_img)
8
9         expected_shape = (1, 224, 224, 3)
10        self.assertEqual(
11            processed.shape, expected_shape,
12            f"Forme attendue {expected_shape}, obtenue {processed.shape}"
13        )
14        self.assertEqual(
15            processed.dtype, np.float32,
16            "Le type de donnees doit etre float32"
17        )
18
19    def test_preprocess_different_input_sizes(self):
20        """Le preprocessing doit fonctionner pour toute taille d'entree."""
21        test_sizes = [(100, 100), (300, 200), (800, 600), (1024, 768)]
22        for width, height in test_sizes:
23            with self.subTest(size=(width, height)):
24                img = Image.new('RGB', (width, height), color='green')
25                processed = preprocess_from_pil(img)
26                self.assertEqual(
27                    processed.shape, (1, 224, 224, 3),
28                    f"Taille {width}x{height} mal redimensionnee"
29                )
```

C — Tests unitaires : détection de la double normalisation

```
1 class TestPreprocessNormalization(unittest.TestCase):
2     """Verifie que le preprocessing ne normalise pas les pixels,
3     la couche Rescaling du modele s'en chargeant."""
4
5     def test_no_manual_normalization(self):
6         """Les valeurs de sortie doivent rester dans [0, 255], pas [0, 1]."""
7         test_img = Image.new('RGB', (224, 224), color=(255, 255, 255))
8         processed = preprocess_from_pil(test_img)
9
10        # Si / 255.0 est present -> max ~= 1.0
11        # Si absent (correct) -> max ~= 255.0
12        self.assertGreater(
13            processed.max(), 1.0,
```

```

14         "Les pixels ne doivent PAS etre normalises dans le preprocessing "
15         "(la couche Rescaling du modele s'en charge)."
```

```

16     )
17     self.assertAlmostEqual(
18         float(processed.max()), 255.0, delta=1.0,
19         msg="Les pixels blancs doivent valoir ~255 apres preprocessing."
20     )
21
22     @unittest.skip("Requiert le vrai modele Keras - executer localement")
23     def test_rescaling_layer_present_in_model(self):
24         """Verifie que le modele contient bien une couche Rescaling(1/255)."""
25         rescaling_layers = [
26             l for l in model.layers
27             if l.__class__.__name__ == 'Rescaling'
28         ]
29         self.assertGreater(len(rescaling_layers), 0,
30             "Le modele doit contenir au moins une couche Rescaling.")
31         scale = rescaling_layers[0].get_config().get('scale', None)
32         self.assertAlmostEqual(scale, 1 / 255.0, places=6,
33             msg="La couche Rescaling doit avoir scale=1/255.")
34
35     @unittest.skip("Requiert le vrai modele Keras - executer localement")
36     def test_no_double_normalization_effect(self):
37         """Deux images differentes doivent produire des predictions differentes.
38         Predictions identiques = symptome de double normalisation."""
39         img1 = Image.open("images_to_test/desert_96.jpg")
40         img2 = Image.open("images_to_test/meadow_89.jpg")
41         pred1 = model.predict(preprocess_from_pil(img1), verbose=0)
42         pred2 = model.predict(preprocess_from_pil(img2), verbose=0)
43         self.assertFalse(
44             np.allclose(pred1, pred2, atol=1e-3),
45             "Predictions identiques = symptome de double normalisation."
46         )

```

D — Captures d'écran Render

Les deux captures ci-dessous documentent le déploiement du commit a958c7f (*fix : rate_limit conditionnel*) sur Render, le 17 mars 2026 à 14h39. Le service est déclaré **Live** et accessible à l'URL de production.

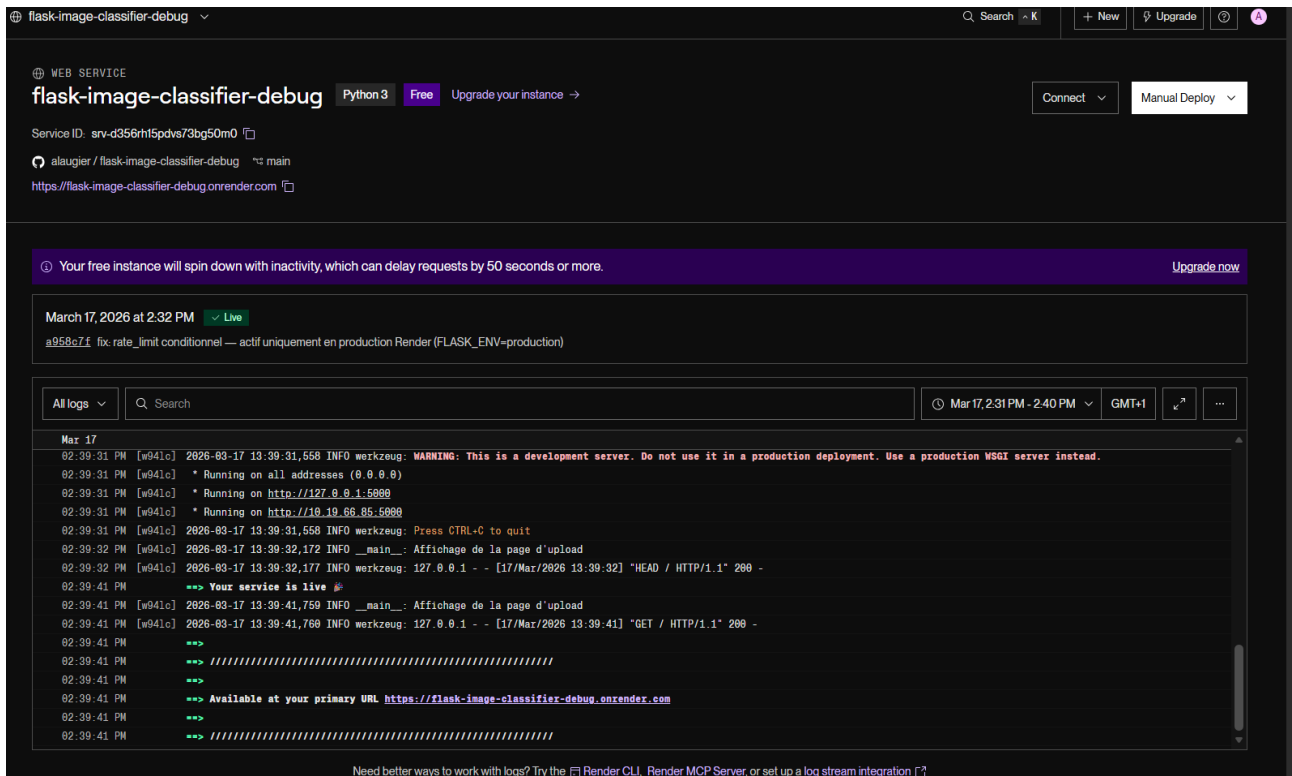


FIGURE 2 – Dashboard Render — dernier déploiement réussi avec logs de démarrage (Your service is live, 17 mars 2026).

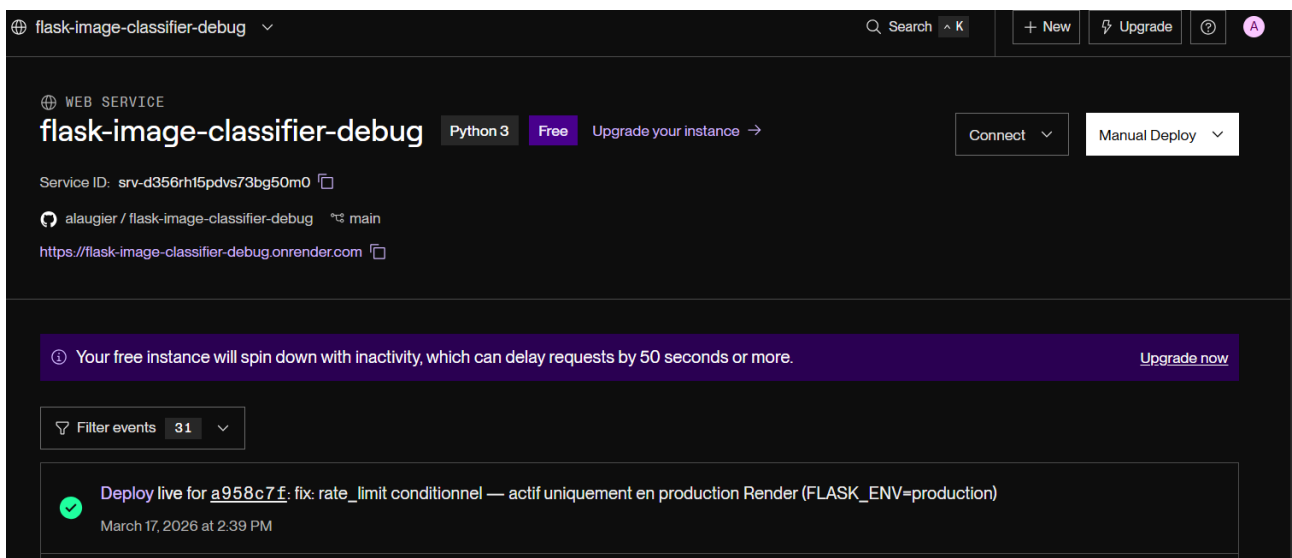


FIGURE 3 – Dashboard Render — statut du service : Deploy live pour le commit a958c7f, branche main.

E — Logs de déploiement Render

Extrait des logs Render du 17 mars 2026 (commit a958c7f), fenêtre 14h31–14h40 GMT+1 :

```
02:39:31 PM * Running on all addresses (0.0.0.0)
02:39:31 PM * Running on http://127.0.0.1:5000
02:39:31 PM * Running on http://10.19.66.85:5000
02:39:32 PM INFO __main__: Affichage de la page d'upload
02:39:32 PM INFO werkzeug: 127.0.0.1 -- "HEAD / HTTP/1.1" 200
02:39:41 PM ==> Your service is live
02:39:41 PM INFO __main__: Affichage de la page d'upload
02:39:41 PM INFO werkzeug: 127.0.0.1 -- "GET / HTTP/1.1" 200
02:39:41 PM ==> Available at your primary URL
https://flask-image-classifier-debug.onrender.com
```

F — Stack technique complète

Composant	Technologie	Rôle
Backend	Flask 3.1.2 [7]	Routing, templates Jinja2, API REST
ML/IA	Keras 3.11.3 [2] + PyTorch (CPU)	Inférence CNN pré-entraîné
Persistance	MongoDB Atlas [6]	Feedbacks utilisateurs (NoSQL)
Monitoring	SQLite + Flask-MonitoringDashboard [1]	Métriques de performance
Tests	unittest (stdlib) [9]	15 tests automatisés, pipeline CI/CD
Déploiement	Gunicorn [5] + Render (PaaS)	Serveur de production

G — Résultats détaillés de la suite de tests

Catégorie de test	Résultat
Validation des extensions de fichier	✓ 5/5 réussis
Redimensionnement et prétraitement	✓ 3/3 réussis
Fonctionnement des routes Flask	✓ 3/3 réussis
Feedback utilisateur	✓ 1/1 réussi
Intégration modèle (mock)	✓ 1/1 réussi
Détection double normalisation	✓ 1/1 réussi
Tests d'intégration vrai modèle (local only)	● 2 sautés en CI
Total CI (durée : 18,4 s)	13/13 réussis + 2 sautés

H — Rate limiting conditionnel (production uniquement)

```
@app.route("/predict", methods=["POST"])
def predict():
    ...

# Rate limiting conditionnel - production uniquement
if os.getenv('FLASK_ENV') == 'production':
    predict = rate_limit(10)(predict)
```